organizations quickly discover that managing permissions in this manner becomes brittle and error-prone.

Importantly, ACLs evaluate access solely based on the principal's identity (or their group membership) and the permissions listed in the ACL. They do not consider additional context, such as time of day, device posture, network zone, or other environmental signals. This simplicity makes them efficient and predictable, but it also makes them inflexible in environments where access decisions must adapt to changing conditions.

These limitations often lead organizations to adopt more flexible access control models—ones that can express broader policies, adapt to dynamic context, and scale more effectively as complexity grows.

5.1.4 Where ACLs still make sense

Despite their limitations, ACLs remain widely used in real systems, and for good reason. In the right context, their simplicity and directness can be strengths, not weaknesses. ACLs are especially well-suited to environments where:

- The set of resources is clearly defined and comparatively stable.
- The number of principals is small or bounded.
- Ownership and access boundaries are clear.
- Policy complexity is low.

Many common tools and platforms still use ACLs as their primary or default authorization model:

- Operating systems such as Unix and Windows use ACLs to manage access to files, directories, and devices.
- Network storage systems (e.g., NFS, SMB) use ACLs to control access to shared drives and folders.
- Databases like PostgreSQL and MySQL use ACL-style grant statements to assign privileges to users and roles for specific tables or schemas.
- Messaging platforms such as RabbitMQ and Kafka often use ACLs to control who can publish to or consume from specific topics or queues.